ECOFLEET AI: INTELLIGENT ELECTRIC VEHICLE FLEET MONITORING AND ROUTE OPTIMIZATION PLATFORM

Submitted in the partial fulfilment for the Course of

CSA1016–SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

CSE WITH AI

Submitted by

KENISHA JUDITH J (192572071)

Under the Supervision of

ANITADAVAMANI K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

1.PROJECT OVERVIEW

1.1 Introduction

The transportation industry is rapidly adopting electric vehicles to reduce carbon emissions, lower fuel dependency, and support sustainable mobility. However, managing electric vehicle fleets requires continuous monitoring of battery health, energy consumption, charging requirements, vehicle availability, and route efficiency.

EcoFleet AI: Intelligent Electric Vehicle Fleet Monitoring and Route Optimization Platform is designed as a smart fleet management system for monitoring electric vehicles and supporting operational decision-making.

The platform integrates a web-based frontend with a Python Flask backend. It provides fleet monitoring, battery analysis, charging recommendations, route optimization, and fleet analytics through an interactive dashboard.

1.2 Problem Statement

Electric vehicle fleet operators need to manage battery performance, charging schedules, vehicle availability, energy consumption, and route planning simultaneously. Traditional fleet monitoring systems may provide vehicle tracking but do not necessarily provide intelligent analysis for battery degradation, charging recommendations, and energy-efficient routing.

Poor route planning can result in unnecessary charging requirements, increased energy consumption, higher operational costs, and delivery delays. Therefore, EcoFleet AI aims to provide an integrated platform that monitors fleet status, analyzes battery conditions, provides charging recommendations, optimizes routes, and presents fleet analytics.

1.3 Objectives

The main objectives of EcoFleet AI are:

1. Develop an AI-powered platform for monitoring electric vehicle fleet performance.
2. Monitor battery health, energy consumption, and vehicle status.
3. Analyze battery degradation and maintenance requirements.
4. Provide intelligent charging recommendations.
5. Recommend energy-efficient routes.
6. Provide a real-time fleet monitoring dashboard.
7. Generate fleet performance analytics.
8. Improve fleet productivity while reducing energy consumption and operational costs.

2. SYSTEM FUNCTIONALITY

The implemented EcoFleet AI platform contains the following major functions:

2.1 Fleet Monitoring

The dashboard displays information about the fleet, including:

- Vehicle ID
- Battery percentage
- Energy consumption
- Vehicle status
- Location
- Vehicle health

The current implementation displays information for 12 electric vehicles.

2.2 Battery Analysis

The battery analysis module retrieves vehicle information and provides:

- Battery level
- Battery health
- Predicted degradation
- Maintenance risk
- Maintenance recommendation

2.3 Charging Recommendation

The charging module analyzes vehicle battery information and provides charging recommendations for the fleet.

2.4 Route Optimization

The route optimization module provides:

- Recommended route
- Distance
- Energy consumption
- Optimization reason

2.5 Fleet Analytics

The dashboard provides six analytics indicators:

- Average Battery
- Average Health
- Average Energy
- Charging Vehicles
- Maintenance Vehicles
- Fleet Efficiency

3. PROJECT STRUCTURE

The EcoFleet AI project is organized into separate directories to maintain a clear and modular software structure. The project separates the backend, frontend, data, and testing components, making the application easier to develop, maintain, and manage using Git.

The project structure is shown below:

3.1 Backend

The **backend** directory contains the Python Flask application responsible for implementing the server-side functionality of EcoFleet AI.

The app.py file contains the Flask application and API endpoints used by the frontend. These APIs provide fleet information, battery analysis, charging recommendations, route optimization, and fleet summary information.

The requirements.txt file contains the Python dependencies required to execute the backend application.

3.2 Frontend

The **frontend** directory contains the user interface of the EcoFleet AI platform.

- **index.html** – Defines the structure and content of the dashboard.
- **style.css** – Defines the visual appearance and layout of the dashboard.
- **script.js** – Handles interaction with the backend APIs and dynamically displays fleet information and analytics.

The frontend communicates with the Flask backend using HTTP requests and JavaScript fetch() functions.

3.3 Data

The **data** directory contains the vehicle-related dataset used by the application.

The dataset provides information such as:

- Vehicle ID
- Battery level
- Energy consumption
- Vehicle status

- Location
- Vehicle health

This data is used by the backend to generate fleet monitoring and analytical results.

3.4 Tests

The **tests** directory is reserved for testing the application functionality. It provides a structured location for unit tests and integration tests that can be added as the project develops.

3.5 README.md

The README.md file provides documentation about the EcoFleet AI project. It can contain information about the project purpose, installation requirements, project structure, execution instructions, and technologies used.

3.6 Modular Project Organization

The separation of the application into backend, frontend, data, and testing directories improves maintainability and makes it easier for developers to work on individual components without affecting the entire application.

This structure also supports version control using Git and provides a suitable foundation for future Docker-based deployment.

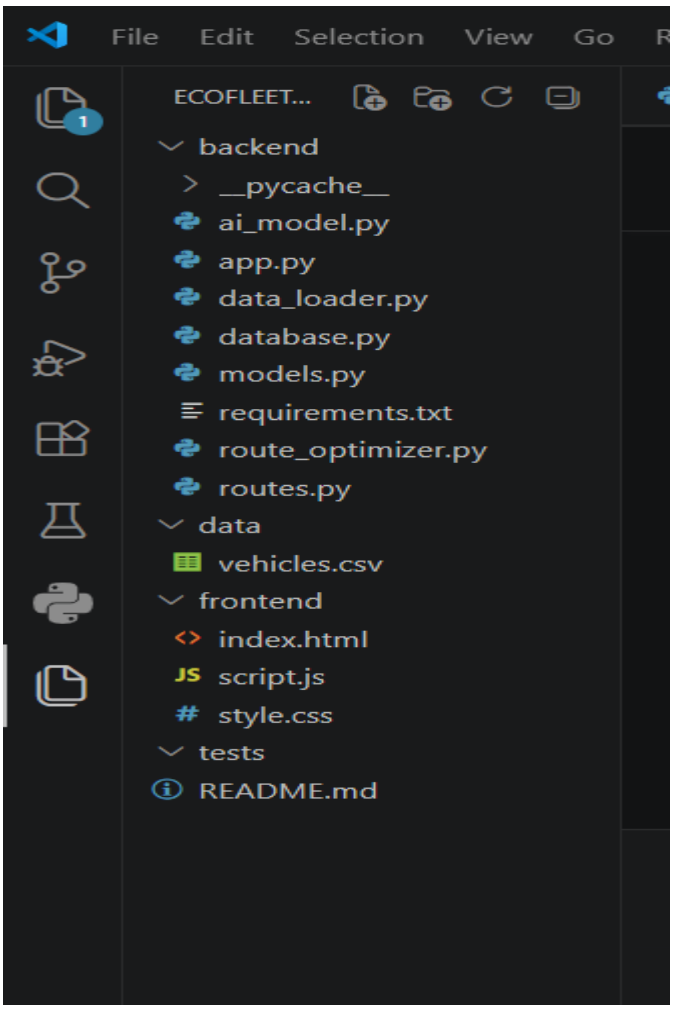


Figure 3.1: EcoFleet AI Project Directory Structure

4. GIT REPOSITORY INITIALIZATION

Git was used as the version control system for the EcoFleet AI project. It provides a systematic way to track source-code changes, maintain different versions of the project, and support collaboration during development.

4.1 Git Installation

Git was installed on the development system and its installation was verified using the following command:

git --version

The command displayed the installed Git version, confirming that Git was successfully configured.

4.2 Git Repository Initialization

The EcoFleet AI project directory was opened in Visual Studio Code. A Git repository was initialized using:

git init

This created a local Git repository for tracking the project files.

4.3 Checking Repository Status

The repository status was checked using:

git status

Initially, the project files were displayed as untracked files. These included the backend, frontend, data, and README components.

4.4 Adding Project Files

All project files were added to Git using:

git add .

This staged the project files so that they could be included in the first commit.

4.5 Creating the Initial Commit

The first commit was created using:

git commit -m "Initial EcoFleet AI project"

The commit stores the initial working version of the EcoFleet AI application.

The commit message is descriptive and clearly indicates that it represents the initial version of the project.

4.6 GitHub Repository

A GitHub repository named **EcoFleet-AI** was created to store the project remotely.

The local repository was connected to GitHub using the remote repository URL:

git remote add origin <GitHub-Repository-URL>

The remote connection was verified using:

git remote -v

The project branch was then renamed to main:

git branch -M main

Finally, the project was uploaded to GitHub using:

git push -u origin main

This successfully synchronized the local EcoFleet AI project with the remote GitHub repository.

4.7 Importance of Git Repository

Git provides the following benefits for the project:

- Tracks changes made to source code.
- Maintains previous versions of the project.
- Provides a history of development activities.
- Supports branching and merging.
- Enables remote backup through GitHub.
- Improves collaboration and project maintainability.

5. GIT BRANCHING STRATEGY

For the EcoFleet AI project, Git branching can be used to separate stable application code from ongoing development and individual feature development.

The proposed branching structure is:

```
main
 |
 └── development
      |
      ├── feature/dashboard
      ├── feature/battery-analysis
      └── feature/docker
```

The **main** branch represents the stable version of the project. The **development** branch is used for ongoing development. Feature branches can be created for individual functionalities without directly modifying the stable branch.

5.1 Main Branch

The main branch contains the stable version of EcoFleet AI that is suitable for demonstration or release.

5.2 Development Branch

The development branch is intended for integrating and testing new changes before they are included in the main branch.

5.3 Feature Branches

Feature branches allow individual functionalities to be developed independently. Examples include:

- feature/dashboard
- feature/battery-analysis
- feature/docker

This approach reduces the possibility of unstable changes affecting the main project.

5.4 Advantages of the Branching Strategy

The branching strategy:

1. Separates stable and development code.
2. Supports parallel development.
3. Reduces conflicts between features.
4. Makes testing easier.
5. Allows changes to be reviewed before merging.
6. Improves project organization and maintenance.

6. VERSION CONTROL WORKFLOW

Git was used to manage the source code and maintain different versions of the EcoFleet AI project. The version control workflow provides a systematic process for adding, committing, and synchronizing project changes.

6.1 Working Directory

The development process begins with the project files in the local EcoFleet AI working directory. Changes to the frontend, backend, data, or documentation are first made in the working directory.

6.2 Staging Changes

After completing a set of changes, the files are added to the Git staging area using:

```
git add .
```

The staging area allows the developer to select the changes that should be included in the next commit.

6.3 Committing Changes

The staged changes are saved permanently in the local Git history using a meaningful commit message.

Example:

```
git commit -m "Initial EcoFleet AI project"
```

The commit message describes the purpose of the change and makes the project history easier to understand.

6.4 Synchronizing with GitHub

The local repository is connected to the remote GitHub repository. Changes can be uploaded using:

git push

This keeps the remote GitHub repository synchronized with the local project.

6.5 Version Control Workflow

The overall workflow can be represented as:

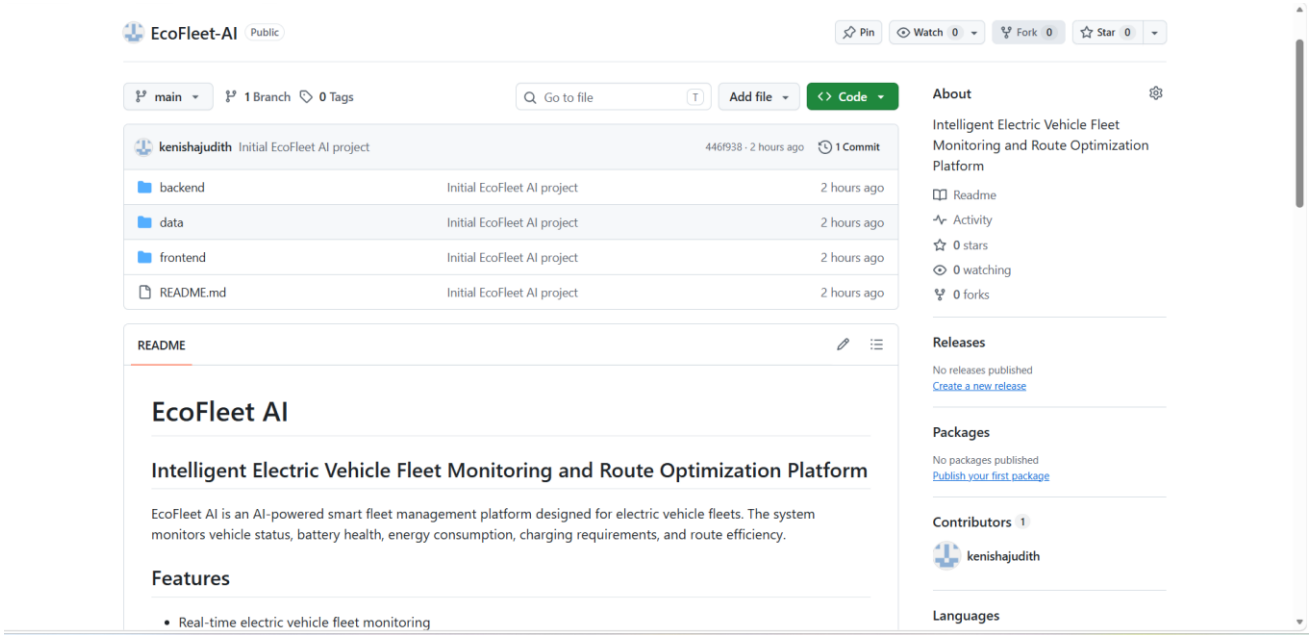
Working Directory
↓
git add .
↓
Staging Area
↓
git commit
↓
Local Repository
↓
git push
↓
GitHub Repository

This workflow ensures that changes are recorded systematically and that the project has a remote copy on GitHub.

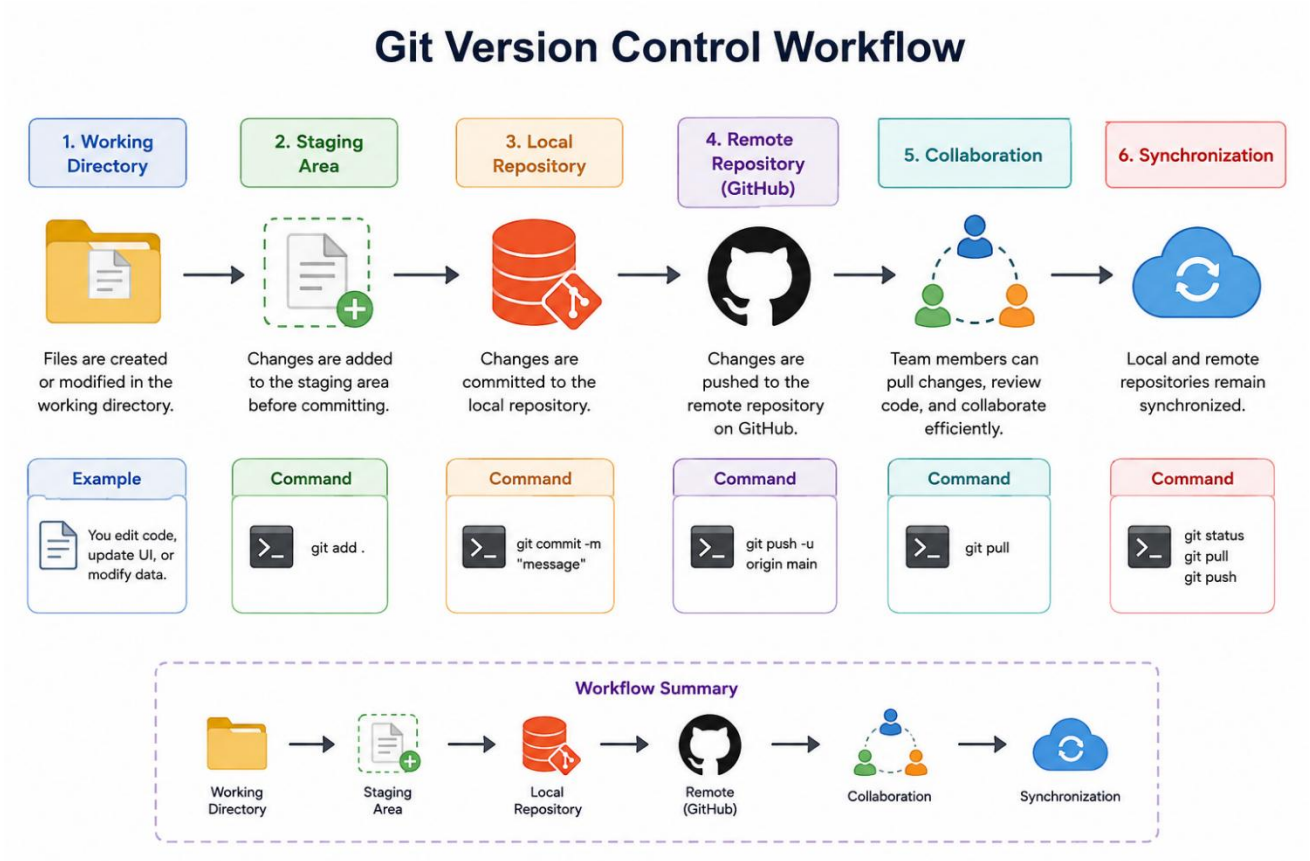
6.6 Good Version Control Practices

The following practices are followed or recommended for the EcoFleet AI project:

- Use meaningful commit messages.
- Commit logical changes rather than unrelated modifications.
- Keep the main branch stable.
- Use development or feature branches for new functionality.
- Review changes before merging.
- Regularly synchronize the local repository with GitHub.
- Avoid committing unnecessary files such as temporary files or generated files.



EcoFleet AI Remote GitHub Repository



7. COMMIT HISTORY ANALYSIS

Git commit history provides a chronological record of changes made to the EcoFleet AI project.

The commit history can be viewed using:

git log --oneline

The initial commit created for the project is:

Initial EcoFleet AI project

This commit represents the initial working version of the project.

7.1 Importance of Commit Messages

Meaningful commit messages help developers understand what was changed and why the change was made. A clear commit history is useful for:

- Tracking project development.
- Identifying changes.
- Debugging problems.
- Reverting unwanted changes.
- Understanding project evolution.
- Supporting collaboration between developers.

7.2 Commit History and Maintenance

A well-maintained Git history makes the EcoFleet AI project easier to maintain because developers can identify previous versions and understand the sequence of development activities.

For future development, commit messages such as:

Add fleet analytics dashboard

Implement battery analysis API

Add charging recommendation module

Implement route optimization

Add Docker configuration

Fix vehicle status display

8. DOCKER ENVIRONMENT DESIGN

8.1 Introduction to Docker

Docker is a containerization platform that packages an application together with its required dependencies, libraries, configuration, and runtime environment into lightweight containers.

For the EcoFleet AI project, Docker can provide a consistent environment in which the application can be developed, tested, and deployed without depending heavily on the configuration of the host computer.

8.2 Why Docker is Suitable for EcoFleet AI

Docker is suitable for EcoFleet AI because the application contains multiple components, including a Python Flask backend and a web-based frontend.

The main advantages of using Docker are:

- **Portability:** The application can run on different systems with the same container configuration.
- **Consistency:** The development and deployment environments can use the same dependencies.
- **Dependency Management:** Required Python packages can be installed inside the container.
- **Isolation:** Application components are isolated from the host operating system.
- **Easy Deployment:** The application can be packaged and deployed more easily.
- **Maintainability:** Container configurations can be version-controlled along with the source code.
- **Scalability:** Additional application containers can be created when required.
-

8.3 Components Requiring Containerization

The following components of EcoFleet AI can be containerized:

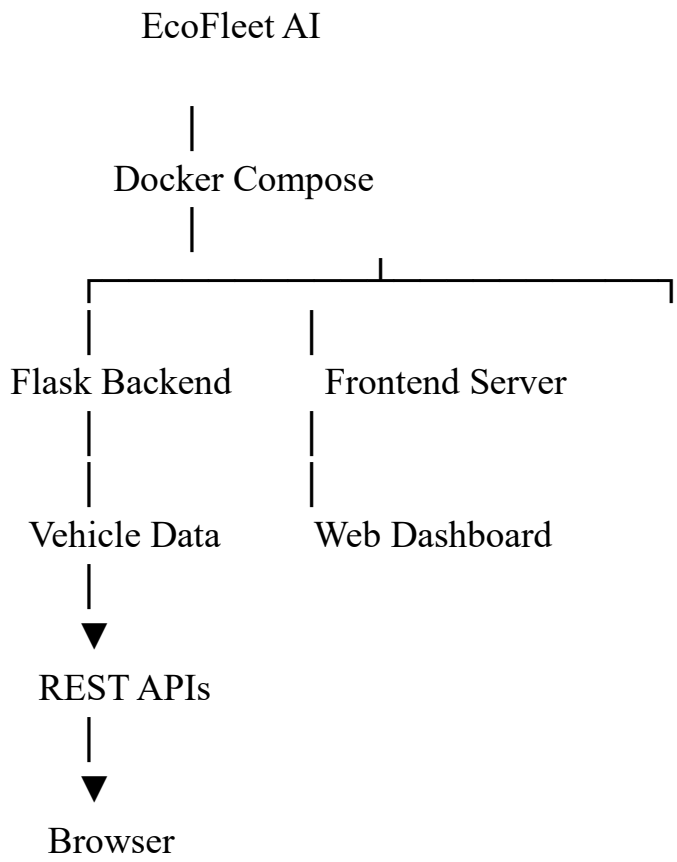
Component	Purpose	Containerization
Flask Backend	Provides APIs and application logic	Required
Frontend	Provides the web dashboard	Recommended
Vehicle Dataset	Provides vehicle information	Mounted as volume
Configuration	Stores environment-specific settings	Environment variables
Database	Stores persistent application data if introduced	Separate container

For the current implementation, the **Flask backend** is the primary component requiring containerization because it contains the application logic and API services.

The frontend can either be served through a lightweight web server container or integrated into the backend deployment.

8.4 Proposed Docker Architecture

The proposed Docker architecture for EcoFleet AI is:



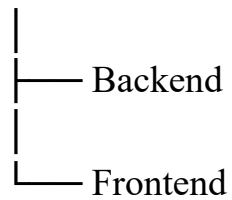
Docker Compose can be used to coordinate the backend and frontend services.

8.5 Docker Network

The Docker services can communicate through a dedicated Docker network.

For example:

EcoFleet Network



This allows the application components to communicate using Docker service names rather than depending on the host machine's configuration.

8.6 Volumes

Docker volumes can be used when persistent data needs to be retained outside the lifecycle of a container.

For EcoFleet AI, a volume can be used for:

- Vehicle datasets

- Database files, if a database is introduced
- Persistent application data

This prevents important data from being lost when a container is recreated.

8.7 Environment Variables

Environment variables can be used to store configuration values instead of hard-coding them in the application.

Examples include:

FLASK_ENV

API_URL

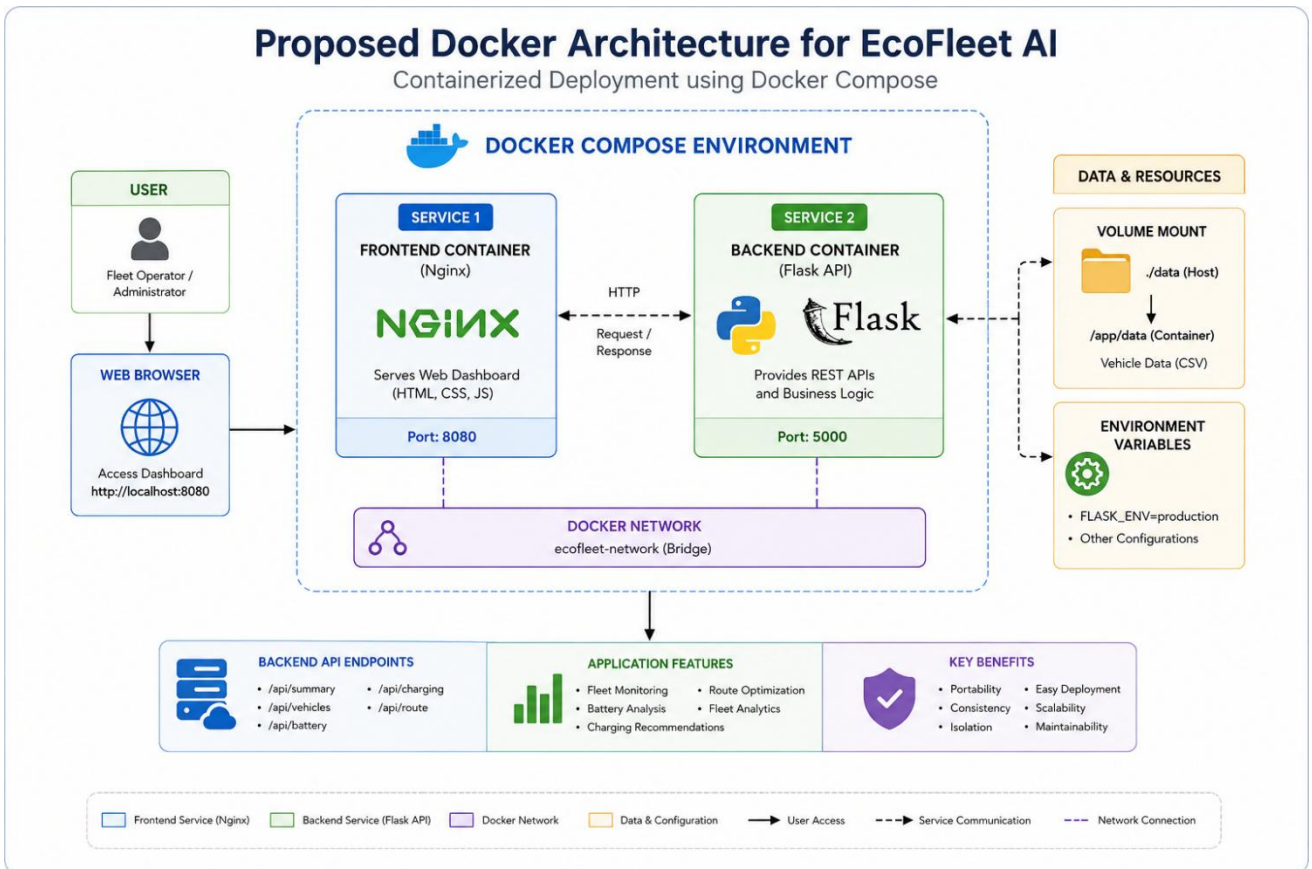
DATABASE_URL

This makes it easier to change configuration between development, testing, and deployment environments.

8.8 Overall Docker Design

The proposed Docker environment provides a standardized deployment structure for EcoFleet AI. Docker Compose can coordinate the different services, while Docker containers provide isolation and reproducibility.

The design can later be extended with a dedicated database, reverse proxy, monitoring service, or additional backend instances if the project requirements increase.



9. DOCKERFILE DEVELOPMENT

A Dockerfile defines the instructions required to build a Docker image for the EcoFleet AI application. It specifies the base Python environment, application files, dependencies, and command required to start the Flask backend.

9.1 Proposed Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY backend/requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY backend/ ./backend/
COPY data/ ./data/

EXPOSE 5000

CMD ["python", "backend/app.py"]
```

9.2 Explanation of Dockerfile Instructions

Instruction	Purpose
FROM	Selects the Python base image.
WORKDIR	Sets the working directory inside the container.
COPY	Copies project files into the container.
RUN	Installs the required Python dependencies.
EXPOSE	Indicates that the Flask application uses port 5000.
CMD	Starts the EcoFleet AI Flask application.

The Dockerfile provides a reproducible environment for running the backend application.

10. DOCKER COMPOSE DESIGN

Docker Compose can be used to define and manage multiple application services using a single configuration file.

10.1 Proposed Docker Compose Configuration

```
services:

  backend:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - ./data:/app/data
    environment:
      - FLASK_ENV=production

  frontend:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./frontend:/usr/share/nginx/html:ro
    depends_on:
      - backend
```

10.2 Services

The configuration contains two main services:

- **Backend:** Runs the Python Flask application and provides the EcoFleet AI APIs.
- **Frontend:** Uses Nginx to serve the HTML, CSS, and JavaScript dashboard.

10.3 Networking

Docker Compose creates a common network for the services, allowing the frontend and backend containers to communicate.

10.4 Volumes

The vehicle data directory is mounted as a volume so that the dataset can be accessed by the backend without being permanently embedded inside the container.

10.5 Environment Variables

Environment variables provide configuration values such as the Flask execution environment without hard-coding them into the application.

10.6 Dependencies

The depends_on instruction establishes that the frontend service depends on the backend service.

11. CONTAINER DEPLOYMENT AND TESTING

The Docker-based deployment is proposed as the deployment environment for EcoFleet AI.

The expected deployment process is:

Dockerfile

↓

Build Docker Image

↓

Docker Compose

↓

Start Containers

↓

Backend + Frontend

↓

Open Dashboard

↓

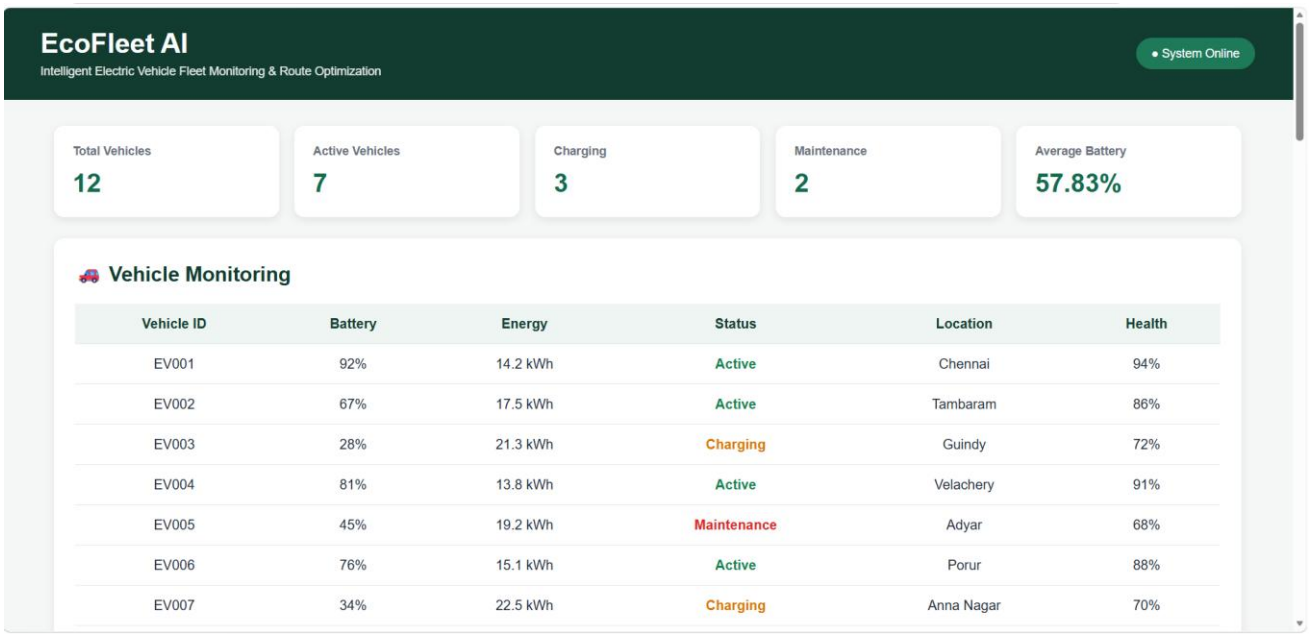
Test APIs and Features

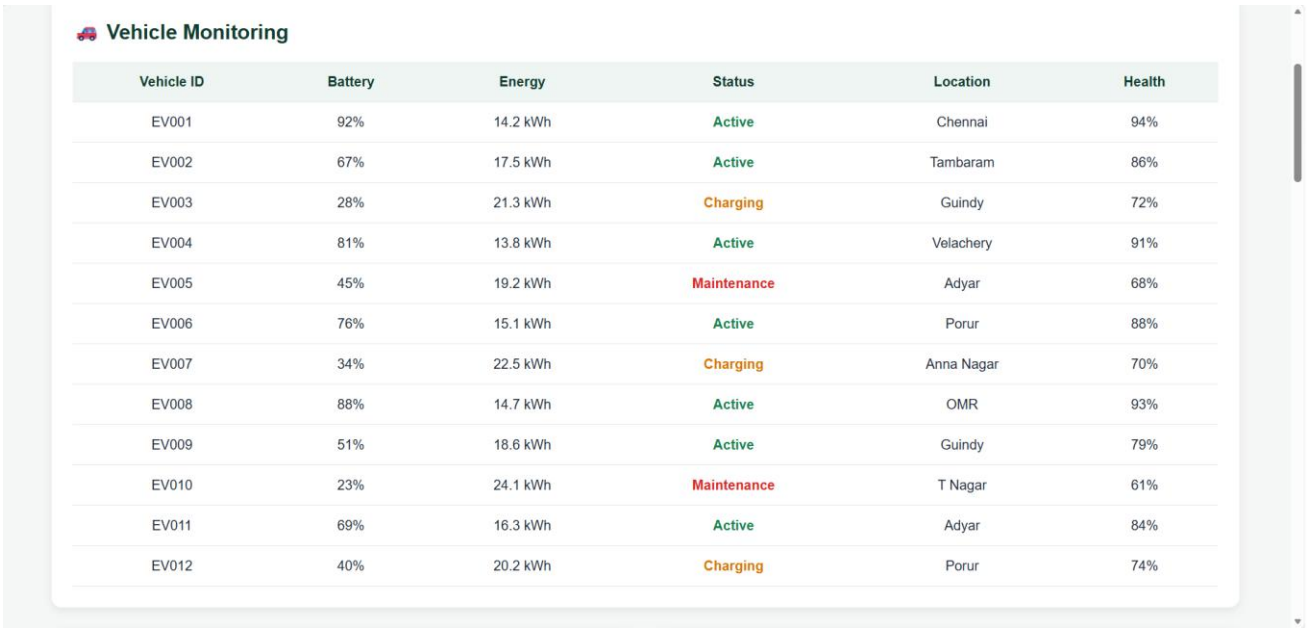
11.1 Testing Procedure

The following tests can be performed after Docker deployment:

Test	Expected Result
Open dashboard	Dashboard loads successfully
Vehicle monitoring	Vehicle information is displayed
Battery analysis	Battery condition is displayed
Charging recommendation	Charging recommendation is generated
Route optimization	Recommended route is displayed
Fleet analytics	Fleet statistics are displayed
Backend API	API returns valid data

The existing EcoFleet AI dashboard was successfully tested during development, with vehicle monitoring, battery analysis, charging recommendations, route optimization, and fleet analytics functioning through the Flask APIs.





Recommended Route: Route B
Distance: 25 km
Energy Consumption: 13 kWh
Optimization Reason: Lowest energy consumption

EcoFleet AI Benefits

Battery Monitoring
Continuous monitoring of EV battery condition.

Smart Charging
Intelligent charging recommendations.

Route Optimization
Energy-efficient route recommendations.

Predictive Maintenance
Early identification of maintenance requirements.

Fleet Analytics

Average Battery	Average Health	Average Energy
57.83%	80.00%	18.12 kWh

Charging Vehicles	Maintenance Vehicles	Fleet Efficiency
3	2	69%

Battery Analysis

EV001 | Analyze Battery

Vehicle: EV001
Battery: 92%
Battery Health: 94%
Predicted Degradation: 6%
Maintenance Risk: Low
Recommendation: Vehicle condition is healthy

Charging Recommendation

EV001
Battery: 92%
Recommendation: **No Charging Required**

EV002
Battery: 67%
Recommendation: **No Charging Required**

EV003
Battery: 28%
Recommendation: **Charge Now**

EV004
Battery: 81%
Recommendation: **No Charging Required**

EV005
Battery: 45%
Recommendation: **Charge Soon**

12. BENEFITS OF GIT AND DOCKER

Git and Docker improve the development and deployment of EcoFleet AI in several ways.

Area	Benefit
Collaboration	Git allows multiple developers to work on the project.
Version Management	Git maintains a history of project changes.
Portability	Docker allows the application to run consistently across systems.
Deployment	Docker simplifies application packaging and deployment.
Scalability	Containers can be replicated when workload increases.
Maintainability	Git history and Docker configuration make the project easier to maintain.
Dependency Management	Docker provides a controlled environment for application dependencies.

Git is mainly responsible for **source-code version management**, while Docker is responsible for **application environment and deployment consistency**.
Together, they provide a reliable development workflow for the EcoFleet AI platform.

13. CHALLENGES AND IMPROVEMENTS

13.1 Challenges Encountered

During implementation, several challenges were encountered:

1. Configuring Git correctly and connecting the local project to GitHub.
2. Managing the separation between frontend and backend components.
3. Connecting JavaScript frontend functions with Flask API endpoints.
4. Ensuring that vehicle data was loaded correctly from the CSV file.
5. Maintaining consistent API responses between the backend and dashboard.
6. Designing the Docker configuration without affecting the existing application.

13.2 Possible Improvements

Future improvements include:

- Implementing automated GitHub Actions for testing.
- Adding proper feature branches and pull requests.
- Creating automated unit and integration tests.
- Containerizing the application using Docker.

- Adding a dedicated database for persistent vehicle information.
- Implementing real-time vehicle tracking.
- Adding advanced AI-based battery degradation prediction.
- Adding authentication and role-based access control.
- Deploying the application to a cloud platform.

14. CONCLUSION

EcoFleet AI was developed as an intelligent electric vehicle fleet monitoring and route optimization platform. The application integrates fleet monitoring, battery analysis, charging recommendations, predictive maintenance, route optimization, and fleet analytics into a unified dashboard.

Git was used for version control and the project was successfully connected to GitHub, providing a remote repository and version history.

Docker was designed as the proposed containerization solution to improve portability, dependency management, deployment consistency, and maintainability. A Dockerfile and Docker Compose configuration were designed for the Flask backend and frontend components.

Overall, the combination of Git for version management and Docker for deployment and portability provides a strong foundation for further development and production deployment of EcoFleet AI.